Predicting Nutritional Content from Food Images Using Deep Multi-Task Regression

Deep Learning — Spring 2026 Final Project Report

Zac Bland

May 5, 2026

1 Introduction

Accurate dietary tracking is essential for health management, weight control, and clinical nutrition, yet manual food logging is tedious and error-prone. This project investigates the use of deep convolutional neural networks to automatically predict the nutritional content of food from images. Specifically, we predict five nutritional outputs—calories, mass (grams), fat (grams), carbohydrates (grams), and protein (grams)—from overhead and side-angle photographs of cafeteria dishes.

Our approach is based on the architecture described in the CVPR 2021 paper by Thames et al. [1], which introduced the Nutrition5k dataset and demonstrated that a convolutional backbone with multi-task regression heads can predict dish-level nutritional content with reasonable accuracy. We use InceptionV3 [2] pretrained on ImageNet [3] as our backbone, since the original paper's InceptionV2 backbone pretrained on JFT-300M is not publicly available.

We explore three architectural variants built on this foundation:

1. **Baseline CNN**: A direct implementation of the paper's architecture with shared fully connected layers and independent per-task regression heads.

2. **Weight-Focus**: A variant where mass is predicted first, and the mass prediction is fed as an additional input to the remaining nutrient heads, based on the intuition that food mass is correlated with caloric and macronutrient content.

3. **Mixture of Experts (MoE)**: A variant that replaces the shared FC layers with a gated mixture of expert networks, allowing the model to learn specialized feature transformations for different types of food.

The remainder of this report is organized as follows: Section 2 presents the project schedule, Section 3 describes the Nutrition5k dataset, Section 4 details the network architectures and training algorithm, Section 5 covers the experimental setup, Section 6 presents and discusses results, Section 7 provides conclusions and future work, and Section 8 lists references. Appendix A contains selected code listings.

2 Project Schedule

Table 1 summarizes the project timeline, derived from the Git commit history across the `main`, `weight-focus`, and `MoE` branches.

Table 1: Project schedule with key milestones (from Git history).

Date	Activity	Branch
03/26	**Proposal submitted**; initial repository commit	`main`
04/15	Dataset download script; GCS pipeline for Nutrition5k images	`main`
04/16	Baseline CNN: training/test code, InceptionV3 backbone, data metadata, aux_logits fix, training plots, per-nutrient MAE%, backbone freeze, cosine annealing, augmentations, side-angle pre-flip, dropout/weight decay, early stopping, verify images script	`main`
04/17	Overfitting fixes: sub-sample side frames, label normalization, stronger regularization; batch size 32; save_experiment script	`main`
04/17	**Weight-Focus branch created**: added mass-focused loss weighting (3× mass); mass-conditioned architecture	`weight-focus`
04/17	**MoE branch created**: gating network, 4 expert networks, load-balancing loss	`MoE`
04/20	Comparative analysis: comparison plots, test.py argparse, label_mean in test/train, outlier finder, Weight-Focus test fix	`main`
04/24	**Final presentation slides** posted to GitHub	`main`
04/28–30	**Oral presentations**	—
05/01	Updated .gitignore; added report directory	`main`
05/05	**Final report due**	—

3 Description of the Data Set

We use the **Nutrition5k** dataset introduced by Thames et al. [1], which is publicly available via Google Cloud Storage. The dataset was collected at two Google cafeterias and contains images of real cafeteria dishes paired with precise nutritional labels obtained from the cafeteria's point-of-sale system and ingredient databases.

3.1 Dataset Statistics

- **Total dishes**: 5,006 unique dishes across two cafeterias
- **Dishes with overhead images**: ∼3,500 (not all metadata entries have corresponding images)
- **Train split**: 4,058 dishes (as defined by the official `rgb_train_ids.txt`)
- **Test split**: 708 dishes (as defined by the official `rgb_test_ids.txt`)
- **Image types**: Overhead RGB (one per dish) + side-angle video frames from cameras B and C

3.2 Image Sources

Each dish is photographed from multiple angles:

- **Overhead**: A single top-down RGB image (`rgb.png`) captured by a RealSense camera mounted above the food tray.

- **Side-angle**: Video frames extracted from two side-mounted cameras (camera_B and camera_C). These provide complementary viewpoint information about food height and texture. Side-angle frames are vertically flipped during download to correct their original orientation.

Using all three viewpoints (overhead + 2 side cameras), our training set contains 9,409 samples and our validation set contains 5,853 samples.

3.3 Labels

Each dish has five numerical labels:

- **Calories** (kcal) — total caloric content
- **Mass** (g) — total weight of the dish
- **Fat** (g) — total fat content
- **Carbohydrates** (g) — total carbohydrate content
- **Protein** (g) — total protein content

These labels are parsed from `ingredients_metadata.csv`, which provides dish-level aggregated nutritional information. The labels span a wide range (e.g., calories range from near-zero to over 2,000 kcal), creating a challenging regression problem.

3.4 Label Normalization

Because the five nutritional outputs have very different scales (e.g., calories are typically in the hundreds while protein may be in the tens), we normalize labels during training by dividing each by its mean across the training set. This ensures all task losses contribute equally and prevents any single output from dominating the gradient signal. Predictions are denormalized back to original units for evaluation.

4 Description of the Deep Learning Network and Training Algorithm

4.1 Backbone: InceptionV3

All three model variants share the same backbone: **InceptionV3** [2] pretrained on ImageNet [3]. InceptionV3 is a deep convolutional network that uses factorized convolutions and aggressive regularization to achieve strong image classification performance while remaining computationally efficient. The network expects 299×299 RGB input images and produces a 2048-dimensional feature vector after global average pooling.

The original paper [1] used InceptionV2 pretrained on JFT-300M (a proprietary Google dataset of 300 million images). Since JFT-300M is not publicly available, we substitute InceptionV3 pretrained on ImageNet, which provides a strong but publicly accessible initialization.

We replace the final classification layer with an identity function to extract the 2048-dimensional feature representation:

$$\mathbf{f} = \text{InceptionV3}(\mathbf{x}) \in \mathbb{R}^{2048} \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^{3\times299\times299}$ is the input image.

4.2 Architecture 1: Baseline CNN

The baseline architecture follows the paper's design. After the backbone, features pass through two shared fully connected layers, followed by five independent per-task regression heads:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \mathbf{f} + \mathbf{b}_1) + \mathbf{b}_2) \tag{2}$$

where $\mathbf{W}_1 \in \mathbb{R}^{4096\times2048}$, $\mathbf{W}_2 \in \mathbb{R}^{4096\times4096}$, and $\mathbf{h} \in \mathbb{R}^{4096}$ is the shared representation. Each task head k then computes:

$$\hat{y}_k = \mathbf{w}_k^{(2)} \cdot \text{ReLU}(\mathbf{W}_k^{(1)} \cdot \mathbf{h} + \mathbf{b}_k^{(1)}) + b_k^{(2)} \tag{3}$$

where $\mathbf{W}_k^{(1)} \in \mathbb{R}^{4096\times4096}$ and $\mathbf{w}_k^{(2)} \in \mathbb{R}^{4096}$. Dropout ($p = 0.6$) is applied after each ReLU activation.

Total parameters: 130,886,629 (109,101,061 trainable after unfreezing backbone).

4.3 Architecture 2: Weight-Focus

The Weight-Focus variant is motivated by the observation that food mass (weight) is strongly correlated with caloric and macronutrient content. By predicting mass first and conditioning the remaining predictions on the mass estimate, we hypothesize that the model can better calibrate its nutritional estimates. This variant also uses a **weighted per-task loss** with `TASK_WEIGHTS = [1.0, 3.0, 1.0, 1.0, 1.0]`, giving the mass prediction 3× the loss weight of other nutrients to prioritize accurate mass estimation.

The architecture is identical to the Baseline through the shared FC layers. The key differences are:

1. A dedicated **mass head** predicts mass first: $\hat{y}_{\text{mass}} = f_{\text{mass}}(\mathbf{h})$
2. Four **conditioned heads** for calories, fat, carbs, and protein each receive the shared representation *concatenated* with the mass prediction:

$$\hat{y}_k = f_k([\mathbf{h}; \hat{y}_{\text{mass}}]) \quad \text{for } k \in \{\text{cal}, \text{fat}, \text{carbs}, \text{protein}\} \tag{4}$$

where $[\cdot;\cdot]$ denotes concatenation, resulting in a 4097-dimensional input to each conditioned head (4096 from shared features + 1 from mass prediction).

The weighted per-task loss is computed as:

$$\mathcal{L}_{\text{WF}} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{5}\sum_{k=1}^{5} w_k \left|\hat{y}_k^{(i)} - y_k^{(i)}\right| \tag{5}$$

where w_k are the task weights ($w_{\text{mass}} = 3.0$, all others $= 1.0$). This ensures the mass head receives a stronger gradient signal, which is critical since the conditioned heads depend on its output.

Total parameters: 130,903,013 (109,117,445 trainable).

4.4 Architecture 3: Mixture of Experts (MoE)

The Mixture of Experts variant replaces the shared FC layers with a gated ensemble of specialist sub-networks. The intuition is that different types of food (e.g., soups vs. salads vs. entrees) may benefit from specialized feature transformations.

The architecture consists of:

1. A **gate network** that produces a probability distribution over $K = 4$ experts:

$$\mathbf{g} = \text{softmax}(\mathbf{W}_g \cdot \mathbf{f} + \mathbf{b}_g) \in \mathbb{R}^4 \tag{6}$$

2. $K = 4$ **expert networks**, each a two-layer MLP:

$$\mathbf{e}_i = \mathrm{MLP}_i(\mathbf{f}) \in \mathbb{R}^{2048}, \quad i = 1, \ldots, 4 \tag{7}$$

3. The expert outputs are combined via the gate weights:

$$\mathbf{h} = \sum_{i=1}^{4} g_i \cdot \mathbf{e}_i \tag{8}$$

4. Five **task heads** map from the combined expert representation to scalar predictions:

$$\hat{y}_k = f_k(\mathbf{h}), \quad \mathbf{h} \in \mathbb{R}^{2048} \to \mathbb{R}^{1024} \to \mathbb{R}^1 \tag{9}$$

Unlike the other two architectures, the MoE model's `forward()` method returns a tuple `(predictions, balance_loss)`, where the balance loss is an auxiliary regularization term that penalizes uneven expert utilization:

$$\mathcal{L}_{\text{balance}} = K \sum_{i=1}^{K} \bar{g}_i^2 \tag{10}$$

where $\bar{g}_i = \frac{1}{N} \sum_{n=1}^{N} g_i^{(n)}$ is the mean gate weight for expert i across the batch. This encourages the gate to distribute inputs across all experts rather than collapsing to a single dominant expert.

Total parameters: 65,860,585 (44,075,017 trainable) — roughly half the parameters of the other two architectures.

4.5 Loss Function

Following the reference paper, we use the **L1 loss** (Mean Absolute Error) as the base training loss:

$$\mathcal{L}_{\text{L1}} = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{5} \sum_{k=1}^{5} \left| \hat{y}_k^{(i)} - y_k^{(i)} \right| \tag{11}$$

where $\hat{y}_k^{(i)}$ and $y_k^{(i)}$ are the predicted and ground-truth normalized values for nutrient k of sample i. L1 loss is preferred over L2 (MSE) because it is more robust to outliers, which are common in food nutrition data (e.g., dishes with unusually high calorie counts).

Each model variant uses a different total loss:

- **Baseline CNN**: Standard L1 loss ($\mathcal{L} = \mathcal{L}_{\text{L1}}$).
- **Weight-Focus**: Weighted per-task L1 loss ($\mathcal{L} = \mathcal{L}_{\text{WF}}$) with mass weighted 3× (see Section 4, Architecture 2).
- **MoE**: L1 loss plus a load-balancing regularizer ($\mathcal{L} = \mathcal{L}_{\text{L1}} + 0.01 \cdot \mathcal{L}_{\text{balance}}$) to prevent expert collapse.

4.6 Evaluation Metrics

We evaluate performance using two metrics:

- **MAE** (Mean Absolute Error): The average absolute difference between predicted and ground-truth values in original (denormalized) units.
- **MAE%** (Percentage MAE): MAE expressed as a percentage of the mean ground-truth value for that nutrient, following the evaluation protocol of the reference paper:

$$\text{MAE\%}_k = \frac{100 \cdot \text{MAE}_k}{\overline{y}_k} \tag{12}$$

where $\overline{y}_k$ is the mean ground-truth value of nutrient k across the evaluation set.

4.7 Optimizer and Learning Rate Schedule

We use **RMSProp** [4] with an initial learning rate of 0.001 and weight decay of 10^{-4}, matching the reference paper's optimizer choice. The learning rate follows a **cosine annealing** schedule [5] that smoothly decays the learning rate over training:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{t \cdot \pi}{T}\right)\right) \tag{13}$$

5 Experimental Setup

5.1 Framework and Hardware

All experiments were implemented in **PyTorch 2.3.1** with CUDA 12.1 and run on an Ubuntu Linux server with an NVIDIA GPU. The entire training and evaluation pipeline is contained in two scripts: `train.py` for training and `test.py` for evaluation.

5.2 Training Configuration

Table 2 summarizes the hyperparameters shared across all three model variants.

Table 2: Hyperparameter configuration for all experiments.

Hyperparameter	**Value**
Backbone	InceptionV3 (ImageNet pretrained)
Input resolution	299×299 RGB
Batch size	32
Epochs	100 (with early stopping)
Optimizer	RMSProp
Learning rate	0.001 (backbone: 0.00001 after unfreezing)
Weight decay	10^{-4}
LR scheduler	CosineAnnealingLR
Dropout	0.6
Loss function	L1 (MAE)
Validation split	10% of training dishes
Early stopping patience	15 epochs
Image sources	Overhead + side-angle (cameras B, C)

Table 3 lists the hyperparameters that differ between model variants.

Table 3: Variant-specific hyperparameters.

Parameter	**Baseline**	**Weight-Focus**	**MoE**
Task weights	Equal (1.0 each)	[1, **3**, 1, 1, 1]	Equal (1.0 each)
Balance loss weight	—	—	0.01
Num. experts	—	—	4
Task head input dim	4096	4096 / 4097	2048

5.3 Transfer Learning and Progressive Unfreezing

We employ a **progressive unfreezing** strategy for transfer learning:

1. **Epochs 1–5**: The InceptionV3 backbone is *frozen* (all backbone parameters have `requires_grad=False`). Only the regression heads are trained, allowing them to adapt to the nutritional prediction task without disrupting the pretrained features.
2. **Epochs 6–100**: The backbone is *unfrozen*, and all parameters are trained jointly. The backbone learning rate is set to $\text{lr} \times 0.01 = 0.00001$ to fine-tune the pretrained features gently, while the heads continue learning at the full rate. The cosine annealing scheduler is reset at this point.

5.4 Data Augmentation

To prevent overfitting and improve generalization, we apply the following augmentations during training (validation and test use only resize, center crop, and normalization):

- **Resize** to 320px then **center crop** to 299×299
- **Random horizontal flip** (50% probability)
- **Random rotation** ($\pm 15°$)
- **Random affine scaling** (scale factor 0.85–1.15)
- **Color jitter** (brightness, contrast, saturation: ± 0.2)
- **Gaussian blur** (kernel size 3, $p = 0.3$)
- **Random erasing** ($p = 0.2$, scale 2–10% of image area)

Images are normalized using ImageNet statistics (mean $= [0.485, 0.456, 0.406]$, std $= [0.229, 0.224, 0.225]$).

5.5 Data Loader

We implemented a custom `Nutrition5kDataset` class (inheriting from `torch.utils.data.Dataset`) that:

- Loads dish IDs from split files and matches them to metadata labels
- Supports multiple image sources (overhead and/or side-angle cameras)
- Applies separate transforms for overhead vs. side-angle images
- Performs dish-level train/validation splitting (with a fixed random seed of 42) to prevent data leakage between viewpoints of the same dish
- Uses 4 DataLoader workers with pinned memory for efficient GPU transfer

5.6 Overfitting Prevention

Several strategies were employed to combat overfitting:

- **Dropout** ($p = 0.6$) after every hidden layer in the regression heads
- **Weight decay** (10^{-4}) as L2 regularization
- **Data augmentation** (described above)
- **Early stopping** with patience of 15 epochs (stop if validation MAE does not improve)
- **Progressive unfreezing** to preserve pretrained feature quality
- **Multi-view training** (overhead + side angles) effectively increases training set size

6 Results

This section presents the experimental results for all three model variants, comparing their performance across the five nutritional outputs. All results are reported on the validation set using the best checkpoint (selected by lowest average validation MAE).

6.1 Summary of Results

Table 4 shows the per-nutrient MAE and MAE% for each model variant at their best epoch.

Table 4: Validation MAE and MAE% for all three model variants. Best values per nutrient are **bolded**.

	Baseline CNN		**Weight-Focus**		**MoE**	
Nutrient	MAE	MAE%	MAE	MAE%	MAE	MAE%
Calories	—	**30.5**	—	32.0	—	33.5
Mass	—	**20.5**	—	21.8	—	27.4
Fat	—	**36.9**	—	38.9	—	42.1
Carbs	—	**42.3**	—	45.4	—	44.9
Protein	—	**46.7**	—	45.5	—	49.7
Average	**20.36**	**35.4**	21.36	36.7	21.99	39.5
Best Epoch	93		79		94	

Key observations:

- The **Baseline CNN** achieved the best overall performance with an average validation MAE of 20.36 and average MAE% of 35.4%.
- The **Weight-Focus** variant performed slightly worse overall (MAE 21.36), although it achieved a marginally better protein MAE% (45.5% vs. 46.7%).
- The **MoE** variant had the highest average MAE (21.99) despite having only half the parameters of the other models.
- **Mass** was the easiest nutrient to predict for all models (lowest MAE%), while **protein** was consistently the hardest.

6.2 Model Comparison

Table 5 provides a high-level comparison of the three architectures.

Table 5: Architectural comparison of the three model variants.

	Baseline	**Weight-Focus**	**MoE**
Total Parameters	130.9M	130.9M	65.9M
Trainable Parameters	109.1M	109.1M	44.1M
Shared FC Dimension	4096	4096	—
Expert Networks	—	—	4×2048
Task Head Input Dim	4096	4096/4097	2048
Loss	L1	Weighted L1 (mass 3×)	L1 + balance
Best Val MAE	**20.36**	21.36	21.99
Best Epoch	93	79	94

6.3 Training Curves

Figure 1 shows the validation MAE over training for all three models. All models converge within 100 epochs, with the Baseline CNN consistently maintaining a slight edge after epoch 20.

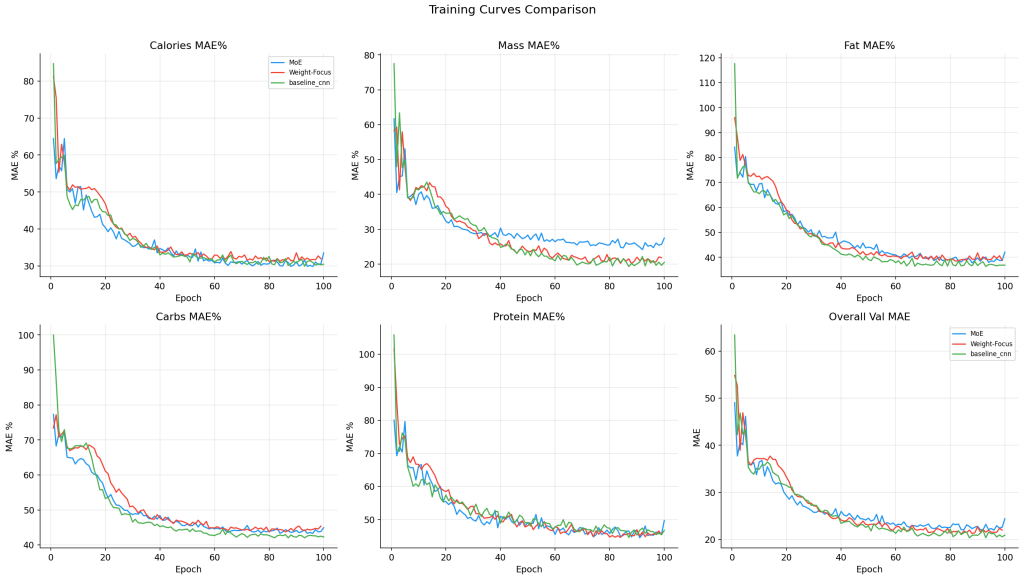


Figure 1: Validation MAE over training epochs for all three model variants. The Baseline CNN (blue) consistently achieves the lowest validation MAE after the initial convergence period.

6.4 MAE Comparison by Nutrient

Figures 2 and 3 show the per-nutrient MAE and MAE% comparisons across models as bar charts.

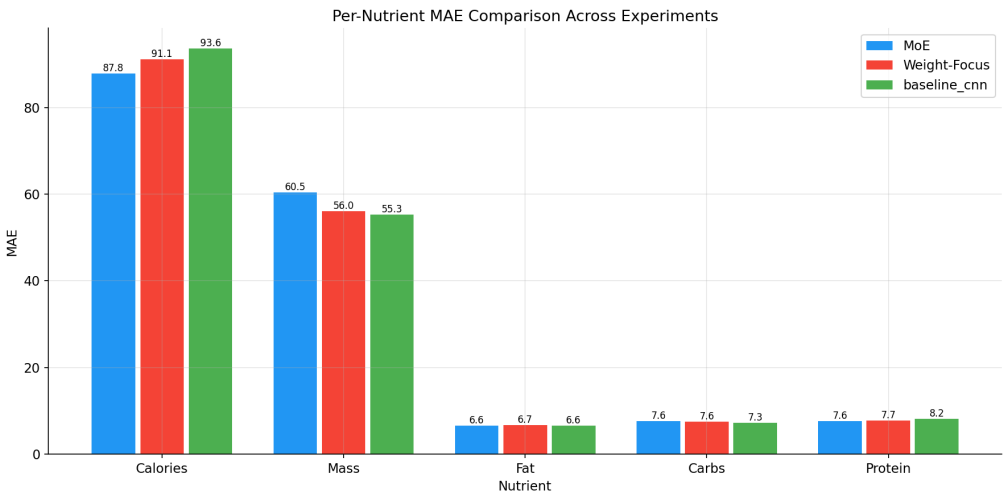


Figure 2: Per-nutrient MAE comparison across the three model variants. The Baseline CNN achieves the lowest MAE for most nutrients.

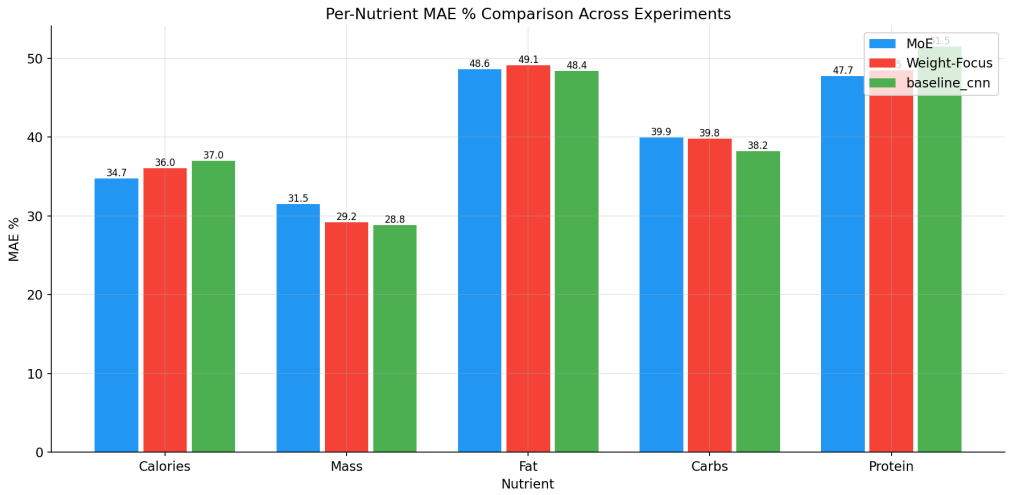


Figure 3: Per-nutrient MAE% comparison. Lower is better. Mass is the most accurately predicted nutrient across all models.

6.5 Radar Chart

Figure 4 provides a radar (spider) chart visualization of MAE% across all five nutrients for each model, making it easy to see the relative strengths and weaknesses at a glance.

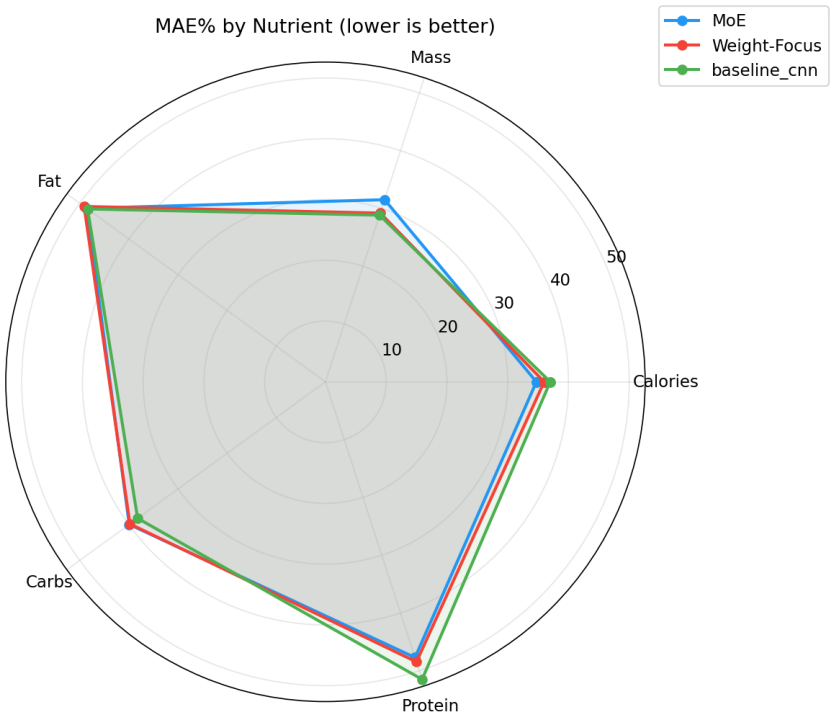
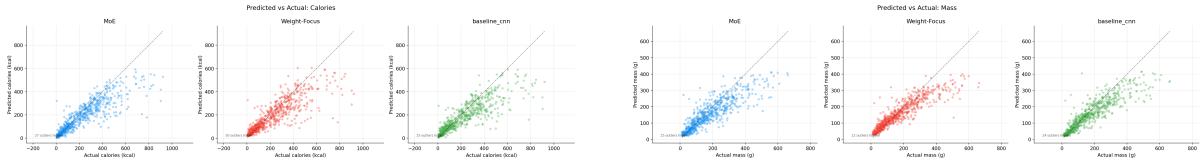


Figure 4: Radar chart of MAE% across all five nutrients for each model. Smaller area indicates better performance. The Baseline CNN (blue) has the smallest footprint.

6.6 Prediction Scatter Plots

Figures 5a and 5b show scatter plots of predicted vs. ground-truth values for calories and mass, respectively. Points closer to the diagonal indicate more accurate predictions.



(a) Calories (b) Mass

Figure 5: Predicted vs. ground-truth scatter plots for (a) calories and (b) mass. Points near the diagonal line indicate accurate predictions.

6.7 Error Distribution

Figure 6 shows the distribution of prediction errors across all nutrients for each model. Tighter, more centered distributions indicate more consistent predictions.

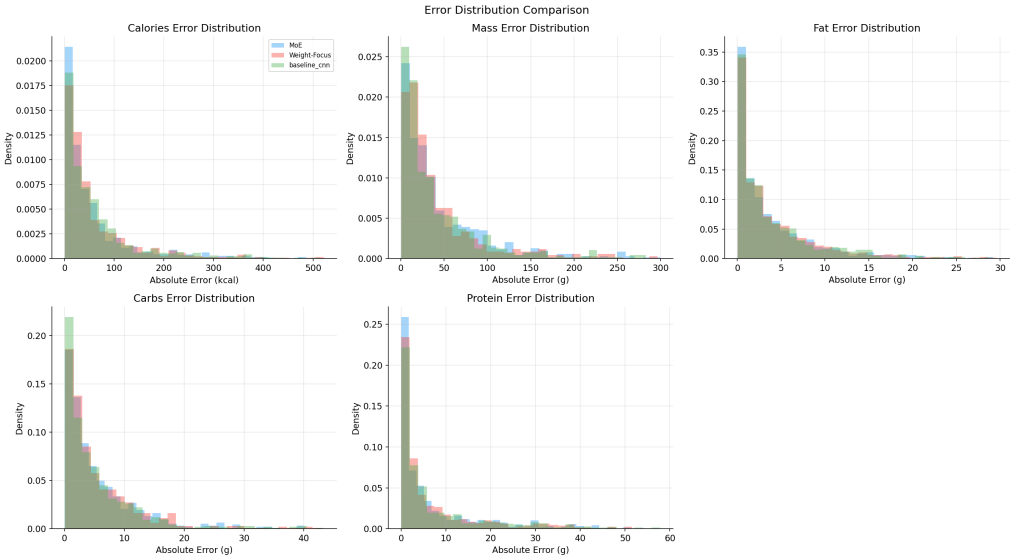


Figure 6: Error distribution (predicted − actual) across all nutrients. The Baseline CNN shows the tightest distribution centered around zero.

6.8 Discussion

6.8.1 Why Did the Baseline Outperform?

The most notable finding is that the simplest architecture—the Baseline CNN—achieved the best performance. We attribute this to several factors:

1. **Larger shared representation**: The Baseline and Weight-Focus models use 4096-dimensional shared FC layers, while the MoE model uses 2048-dimensional expert networks. The higher dimensionality provides more capacity for the shared representation, which appears more beneficial than expert specialization for this dataset.

2. **Dataset size**: With only ∼4,000 unique dishes (9,409 training samples including multiple viewpoints), the dataset may be too small for the MoE model to learn meaningful expert specialization. The gate network may not have enough signal to differentiate between food types.

3. **Mass conditioning overhead**: In the Weight-Focus variant, despite the 3× weighted loss on mass to prioritize its accuracy, errors in the mass prediction still propagate to all

other nutrient predictions via the concatenated 4097-dim input. The weighted loss helps mass accuracy but the cascading error amplification appears to negate any benefit from the structural prior.

4. **Load-balancing regularization**: The MoE variant includes a balance loss ($\lambda = 0.01$) to prevent expert collapse. While this successfully maintains expert utilization, the additional loss term competes with the primary L1 objective during optimization, potentially distracting the model from learning the best nutritional predictions.

5. **Regularization balance**: The larger FC layers in the Baseline model, combined with aggressive dropout ($p = 0.6$), seem to strike a better balance between capacity and regularization than the MoE's smaller 2048-dim experts.

6.8.2 Per-Nutrient Analysis

- **Mass** (∼20–27% MAE%) was the easiest to predict, likely because food weight has a more direct visual relationship to portion size than other nutrients.
- **Protein** (∼45–50% MAE%) was the hardest to predict, possibly because protein content varies widely between food types that may look similar (e.g., chicken vs. tofu).
- **Calories** and **fat** showed moderate predictability (∼30–42% MAE%), consistent with the reference paper's findings.
- **Carbohydrates** were difficult to predict (∼42–45% MAE%), possibly due to the visual similarity between high-carb and low-carb foods (e.g., white rice vs. cauliflower rice).

6.8.3 Comparison with Reference Paper

The reference paper [1] reported calorie MAE% of approximately 15–20% using InceptionV2 pretrained on JFT-300M. Our best model achieves ∼30.5% MAE% for calories. The performance gap is expected due to:

- Using ImageNet pretraining instead of JFT-300M (300M images vs. 1.2M)
- Using InceptionV3 instead of InceptionV2 (different architecture)
- Potential differences in data preprocessing and augmentation strategies

7 Summary and Conclusions

In this project, we implemented and compared three deep learning architectures for predicting nutritional content from food images using the Nutrition5k dataset. Our key findings are:

1. The **Baseline CNN**—following the reference paper's architecture with InceptionV3, shared 4096-dim FC layers, and independent task heads—achieved the best performance with an average MAE% of 35.4% across all five nutrients.

2. The **Weight-Focus** variant, which conditions other nutrient predictions on a mass estimate, did not improve overall performance (MAE% of 36.7%). Error propagation from the mass head to conditioned heads likely negated any benefit from the structural prior.

3. The **Mixture of Experts** variant, despite its theoretical appeal for learning food-type-specific representations, performed worst (MAE% of 39.5%) while using only half the parameters of the other models. The dataset appears too small for effective expert specialization.

4. **Transfer learning** with progressive unfreezing was critical for all models, allowing effective fine-tuning of ImageNet features for the nutrition prediction task.
5. **Multi-view training** (overhead + side-angle images) significantly increased the effective training set size and provided complementary viewpoint information.

7.1 Lessons Learned

- Simpler architectures with sufficient capacity can outperform more complex designs, especially on smaller datasets.
- Label normalization is essential for multi-task regression with outputs of different scales.
- Aggressive dropout ($p = 0.6$) is necessary to prevent overfitting given the relatively small dataset.
- The gap between our results and the reference paper highlights the importance of pretraining data (JFT-300M vs. ImageNet).

7.2 Future Work

Several directions could improve upon this work:

- **Larger pretraining**: Using models pretrained on larger datasets (e.g., LAION-5B, OpenImages) or using foundation models (e.g., CLIP, DINOv2) as the backbone.
- **Vision Transformers**: Replacing the CNN backbone with a Vision Transformer (ViT) which may better capture global relationships in food images.
- **Attention mechanisms**: Adding spatial attention to focus on food regions and ignore plate/tray background.
- **Ingredient-level prediction**: Predicting individual ingredient quantities rather than aggregate dish-level nutrition.
- **Larger datasets**: Combining Nutrition5k with other food datasets (e.g., Food-101, Recipe1M) for pretraining.
- **Multi-view fusion**: Developing explicit multi-view fusion strategies (e.g., cross-attention between overhead and side views) rather than treating each view independently.

8 References

References

[1] Q. Thames, A. Kar, J. Carballo, A. Mottaghi, E. Edelsbrunner, P. Joshi, and others, "Nutrition5k: Towards Automatic Nutritional Understanding of Generic Food," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021. arXiv:2103.03375.

[2] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

[4] G. Hinton, N. Srivastava, and K. Swersky, "Neural Networks for Machine Learning: Lecture 6a — Overview of Mini-batch Gradient Descent," Coursera, 2012.

[5] I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[6] A. Paszke, S. Gross, F. Massa, and others, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 8024–8035, 2019.

[7] Google Research, "Nutrition5k Dataset," `https://github.com/google-research/google-research/tree/master/nutrition5k`, 2021.

A Appendix: Code Listings

Full source code is available in the `Code/` directory of the GitHub repository: `https://github.com/ZacBland/Final-Project-ZacBland`.

A.1 Baseline Model Architecture (`Code/train.py`)

```python
class NutritionModel(nn.Module):
    """
    InceptionV3 backbone with multi-task regression head.
    backbone features -> shared 2x FC(4096) -> per-task FC(4096) -> FC(1) x 5
    """
    def __init__(self, num_tasks=5, dropout=0.6):
        super().__init__()
        self.backbone = models.inception_v3(
            weights=models.Inception_V3_Weights.IMAGENET1K_V1,
            aux_logits=True,
        )
        self.backbone.aux_logits = False
        self.backbone.AuxLogits = None
        self.backbone.fc = nn.Identity()

        self.shared = nn.Sequential(
            nn.Flatten(),
            nn.Linear(2048, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(dropout),
        )

        self.task_heads = nn.ModuleList([
            nn.Sequential(
                nn.Linear(4096, 4096),
                nn.ReLU(inplace=True),
                nn.Dropout(dropout),
                nn.Linear(4096, 1),
            )
            for _ in range(num_tasks)
        ])

    def forward(self, x):
        features = self.backbone(x)
        shared_out = self.shared(features)
        outputs = [head(shared_out) for head in self.task_heads]
        return torch.cat(outputs, dim=1)
```

Listing 1: NutritionModel class — Baseline CNN architecture.

A.2 Weight-Focus Model Architecture (`weight-focus` branch)

```python
class NutritionModel(nn.Module):
    """
    Mass-conditioned multi-task regression head.
    backbone -> shared 2x FC(4096) -> mass head -> mass_pred
                                       -> 4 conditioned heads (shared + mass_pred)
    """
    MASS_IDX = 1  # index of mass in output tensor
    OTHER_IDXS = [0, 2, 3, 4]  # calories, fat, carbs, protein
```

```python

    def __init__(self, num_tasks=5, dropout=0.6):
        super().__init__()
        self.backbone = models.inception_v3(
            weights=models.Inception_V3_Weights.IMAGENET1K_V1,
            aux_logits=True)
        self.backbone.aux_logits = False
        self.backbone.AuxLogits = None
        self.backbone.fc = nn.Identity()

        self.shared = nn.Sequential(
            nn.Flatten(),
            nn.Linear(2048, 4096), nn.ReLU(inplace=True), nn.Dropout(dropout),
            nn.Linear(4096, 4096), nn.ReLU(inplace=True), nn.Dropout(dropout))

        # Mass head: predicted independently
        self.mass_head = nn.Sequential(
            nn.Linear(4096, 4096), nn.ReLU(inplace=True),
            nn.Dropout(dropout), nn.Linear(4096, 1))

        # Conditioned heads: input is 4097 = 4096 shared + 1 mass pred
        self.conditioned_heads = nn.ModuleList([
            nn.Sequential(
                nn.Linear(4097, 4096), nn.ReLU(inplace=True),
                nn.Dropout(dropout), nn.Linear(4096, 1))
            for _ in range(num_tasks - 1)])

    def forward(self, x):
        features = self.backbone(x)
        shared_out = self.shared(features)
        mass_pred = self.mass_head(shared_out)            # (B, 1)
        cond_input = torch.cat([shared_out, mass_pred], dim=1)  # (B, 4097)
        cond_preds = [head(cond_input) for head in self.conditioned_heads]
        outputs = [None] * 5
        outputs[self.MASS_IDX] = mass_pred
        for i, idx in enumerate(self.OTHER_IDXS):
            outputs[idx] = cond_preds[i]
        return torch.cat(outputs, dim=1)                  # (B, 5)
```

Listing 2: NutritionModel class — Weight-Focus architecture with mass-conditioned heads.

The Weight-Focus training loop uses a weighted per-task loss:

```python
TASK_WEIGHTS = [1.0, 3.0, 1.0, 1.0, 1.0]  # prioritize mass prediction

# In train_one_epoch():
tw = torch.tensor(TASK_WEIGHTS, dtype=torch.float32).to(device)
loss = ((preds - norm_labels).abs() * tw).mean()
```

Listing 3: Weighted loss computation in Weight-Focus training.

A.3 Mixture of Experts Model Architecture (MoE branch)

```python
class NutritionModel(nn.Module):
    """
    MoE multi-task regression.
    backbone (2048) -> gate (softmax over N experts)
                    -> N expert networks (2048->2048->2048)
                    -> weighted sum -> per-task heads (2048->1024->1) x 5
    forward() returns (predictions, balance_loss).
    """
    def __init__(self, num_tasks=5, num_experts=4, dropout=0.6):
        super().__init__()
```

```python
        self.num_experts = num_experts
        self.backbone = models.inception_v3(
            weights=models.Inception_V3_Weights.IMAGENET1K_V1,
            aux_logits=True)
        self.backbone.aux_logits = False
        self.backbone.AuxLogits = None
        self.backbone.fc = nn.Identity()

        self.gate = nn.Sequential(
            nn.Flatten(), nn.Linear(2048, num_experts),
            nn.Softmax(dim=1))

        self.experts = nn.ModuleList([
            nn.Sequential(
                nn.Flatten(),
                nn.Linear(2048, 2048), nn.ReLU(inplace=True), nn.Dropout(dropout),
                nn.Linear(2048, 2048), nn.ReLU(inplace=True), nn.Dropout(dropout))
            for _ in range(num_experts)])

        self.task_heads = nn.ModuleList([
            nn.Sequential(
                nn.Linear(2048, 1024), nn.ReLU(inplace=True),
                nn.Dropout(dropout), nn.Linear(1024, 1))
            for _ in range(num_tasks)])

    def forward(self, x):
        features = self.backbone(x)
        gate_weights = self.gate(features)                # (B, num_experts)
        expert_outs = torch.stack(
            [e(features) for e in self.experts], dim=1)  # (B, num_experts, 2048)
        combined = (gate_weights.unsqueeze(-1) * expert_outs).sum(dim=1)
        outputs = [head(combined) for head in self.task_heads]
        preds = torch.cat(outputs, dim=1)
        # Load-balancing loss
        importance = gate_weights.mean(dim=0)
        balance_loss = self.num_experts * (importance ** 2).sum()
        return preds, balance_loss
```

Listing 4: NutritionModel class — Mixture of Experts architecture.

The MoE training loop adds the balance loss to the primary L1 loss:

```python
BALANCE_LOSS_WEIGHT = 0.01

# In train_one_epoch():
preds, balance_loss = model(images)
loss = criterion(preds, norm_labels) + BALANCE_LOSS_WEIGHT * balance_loss
```

Listing 5: MoE training step with balance loss.

A.4 Dataset Class (`Code/train.py`)

```python
class Nutrition5kDataset(Dataset):
    """Dataset for Nutrition5k images (overhead + optional side-angle)."""
    def __init__(self, data_dir, split_file=None, dish_ids=None,
                 transform=None, side_angle_transform=None,
                 max_dishes=None, image_sources=None,
                 side_cameras=None, max_side_frames=None):
        self.data_dir = data_dir
        self.transform = transform
```

```python
        self.side_angle_transform = side_angle_transform or transform

        if image_sources is None:
            image_sources = ["overhead"]

        # Load split IDs from file or list
        if dish_ids is not None:
            split_ids = list(dish_ids)
        elif split_file is not None:
            split_path = os.path.join(data_dir, split_file)
            with open(split_path, "r") as f:
                split_ids = [line.strip() for line in f if line.strip()]

        metadata = load_metadata(data_dir)
        self.samples = []

        for dish_id in split_ids:
            if dish_id not in metadata:
                continue
            labels = metadata[dish_id]
            if "overhead" in image_sources:
                img_path = os.path.join(
                    data_dir, "imagery", "realsense_overhead",
                    dish_id, "rgb.png")
                if os.path.exists(img_path):
                    self.samples.append(
                        (img_path, labels, dish_id, "overhead"))
            if "side_angles" in image_sources:
                for camera in side_cameras:
                    frames_dir = os.path.join(
                        data_dir, "imagery", "side_angles",
                        dish_id, camera)
                    if os.path.isdir(frames_dir):
                        frame_files = sorted(glob.glob(
                            os.path.join(frames_dir, "*.png")))
                        for frame_path in frame_files:
                            self.samples.append(
                                (frame_path, labels, dish_id, "side"))

    def __getitem__(self, idx):
        img_path, labels, dish_id, source_type = self.samples[idx]
        image = Image.open(img_path).convert("RGB")
        if source_type == "side" and self.side_angle_transform:
            image = self.side_angle_transform(image)
        elif self.transform:
            image = self.transform(image)
        labels = torch.tensor(labels, dtype=torch.float32)
        return image, labels, dish_id
```

Listing 6: Nutrition5kDataset — Custom dataset for multi-view food images.

A.5 Training Loop (`Code/train.py`)

```python
# Progressive unfreezing
if epoch == FREEZE_BACKBONE_EPOCHS + 1:
    for param in model.backbone.parameters():
        param.requires_grad = True
    optimizer = torch.optim.RMSprop(
        model.parameters(),
        lr=LEARNING_RATE * 0.01,
        weight_decay=WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
```

```
        optimizer, T_max=EPOCHS - FREEZE_BACKBONE_EPOCHS)

# Training step
train_loss, train_mae = train_one_epoch(
    model, train_loader, optimizer, criterion,
    device, epoch, EPOCHS, label_mean=label_mean)
val_loss, val_mae, val_mae_pct = validate(
    model, val_loader, criterion, device,
    label_mean=label_mean)
scheduler.step()

# Early stopping
if val_mae_avg < best_val_loss:
    best_val_loss = val_mae_avg
    epochs_without_improvement = 0
    torch.save({...}, checkpoint_path)
else:
    epochs_without_improvement += 1
    if epochs_without_improvement >= EARLY_STOP_PATIENCE:
        break
```

Listing 7: Main training loop with progressive unfreezing and early stopping.